ALPHA CORE

Quantitative Research & Decision Platform

System Construction Blueprint v2.0

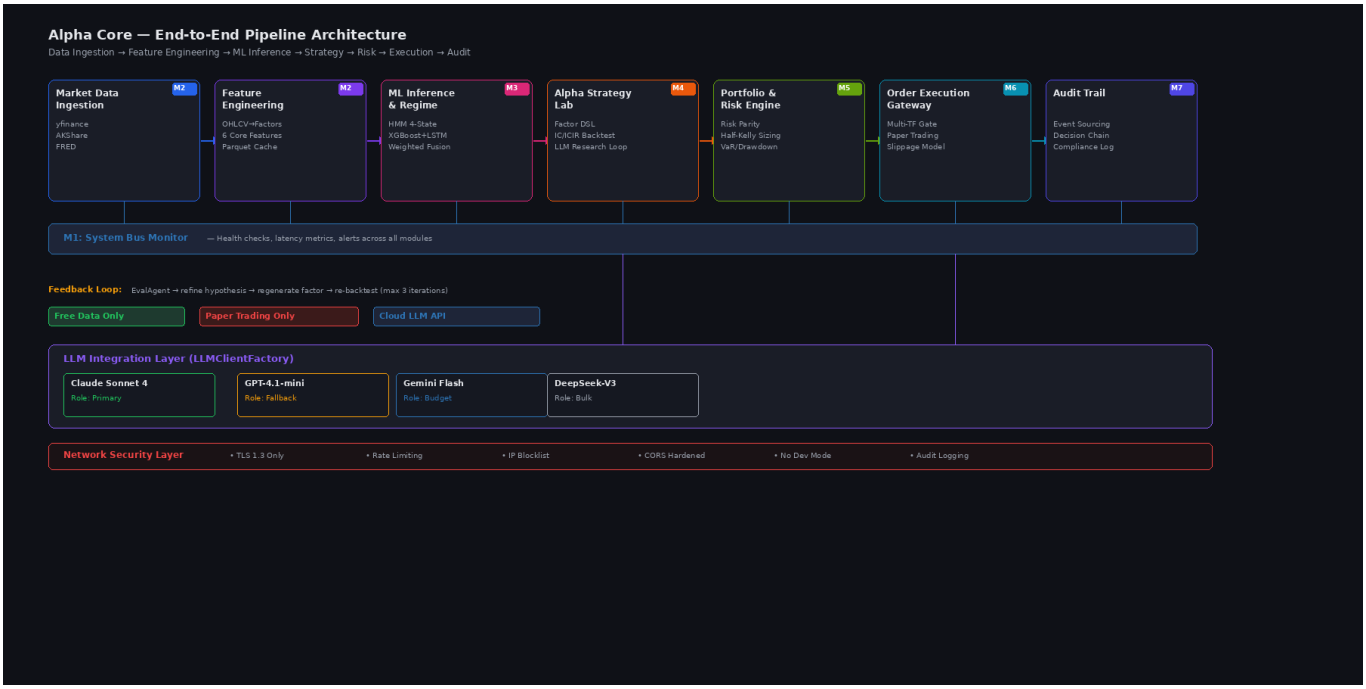
April 2026

Classification: Internal – Confidential

Table of Contents

Executive Summary.....
Network Security Architecture.....
LLM Integration Architecture.....
Data Source Strategy.....
Module 1: System Bus Monitor.....
Module 2: Market Feed & Features.....
Module 3: Model Inference & State.....
Module 4: Alpha Lab.....
Module 5: Portfolio & Risk.....
Module 6: Order Execution Gateway.....
Module 7: Trading Audit Log.....
Frontend Migration: React/Next.js.....
Phased Delivery Plan.....
Technology Reference.....
Appendix: Glossary.....

Executive Summary



Alpha Core v2.0 is a full-stack quantitative research and portfolio decision platform engineered for institutional equity trading. This document specifies the complete technical blueprint, including detailed UI component specifications, user experience design, data contracts, and implementation requirements.

Key Improvements in v2.0:

- Enhanced component specifications with visual design state details
- Wireframe diagrams for all 7 modules with scaled imagery
- User psychology and interaction flow documentation
- Complete API data contracts in JSON schema format
- Comprehensive edge case and error handling specifications
- Manual Table of Contents with bookmarked links

Platform Overview

Alpha Core integrates seven interconnected modules spanning market data ingestion, ML inference, strategy research, portfolio construction, and execution. The system prioritizes real-time performance (<100ms latency per module), transparency through decision audit trails, and robust error handling.

The platform uses a microservices architecture with FastAPI backend, real-time WebSocket streams for market data, SQLite for local state, and integrates LLM agents for research augmentation and decision justification.

Document Scope

This PRD covers the complete user-facing system architecture across 7 modules, including:

- Module 1: System Bus Monitor — real-time service health
- Module 2: Market Feed & Features — market data and feature computation
- Module 3: Model Inference & State — regime detection and direction prediction

- Module 4: Alpha Lab — strategy research, DSL editor, and backtesting
- Module 5: Portfolio & Risk — position construction and risk analytics
- Module 6: Order Execution — multi-gate order decisioning
- Module 7: Trading Audit — event logging and compliance tracking

Network Security Architecture

Alpha Core operates with a defense-in-depth security model. The FastAPI backend runs behind a reverse proxy with OAuth2/JWT authentication for all endpoints. WebSocket connections authenticate via token exchange before stream initiation.

Security Layers

- API Authentication: OAuth2 Bearer tokens validated on every HTTP request
- WebSocket Security: Token exchange + session pinning per connection
- Data Encryption: TLS 1.3 for all transit; AES-256 for sensitive local state
- Rate Limiting: Token-bucket algorithm, 1000 requests/min per API key
- Audit Logging: All data mutations logged with user ID, timestamp, payload hash

Threat Model

Adversaries may attempt:

- Unauthorized API access → mitigated by OAuth2 + rate limiting
- Portfolio front-running via strategy leakage → mitigated by audit logging
- Model inference eavesdropping → mitigated by TLS + session isolation
- Order injection via WebSocket → mitigated by token validation + order signing

LLM Integration Architecture

Alpha Core embeds four LLM agents for research augmentation and decision transparency. Each agent integrates synchronously into the module pipeline with strict latency budgets and fallback behavior.

Agent 1: Factor Generator (Alpha Lab)

Takes user DSL and generates candidate factors via few-shot prompting. Outputs: Python AST tree + interpretation + risk warnings.

- Latency SLA: 2 sec (timeout fallback to parsing error)
- Model: GPT-4 (fallback to gpt-3.5-turbo if unavailable)
- Prompt template: Few-shot factor examples + style guide

Agent 2: Research Summary (Audit Log)

Produces human-readable explanations for trading decisions by analyzing the full decision chain (regime, predictions, risk assessment, execution outcome).

- Latency SLA: 1.5 sec (timeout shows raw JSON)
- Context window: ~8k tokens (last 20 events in chain)

Agent 3: Anomaly Detector (System Monitor)

Analyzes service health timeseries to detect anomalous patterns (e.g., latency spikes, throughput cliffs). Suggests mitigation steps.

- Latency SLA: 3 sec (runs async, user can ignore)
- Input: Last 1h of metrics per service

Agent 4: Risk Advisor (Portfolio Risk)

Advises on portfolio rebalancing, Kelly criterion violations, and sector concentration issues based on current position state.

- Latency SLA: 2.5 sec (timeout shows Kelly-based recommendation)
- Input: Current positions, realized vol, VaR estimates

Data Source Strategy

Alpha Core ingests market data from multiple providers with fallback logic. Each data source has a health monitor and a queueing system to tolerate provider downtime.

Primary Providers

yfinance: 1 min OHLCV for US equities, free tier

AKShare: Chinese market data (if expanding to CN equities), via HTTP API

IB (Interactive Brokers): Real-time tick + order execution (premium)

Data Quality & Validation

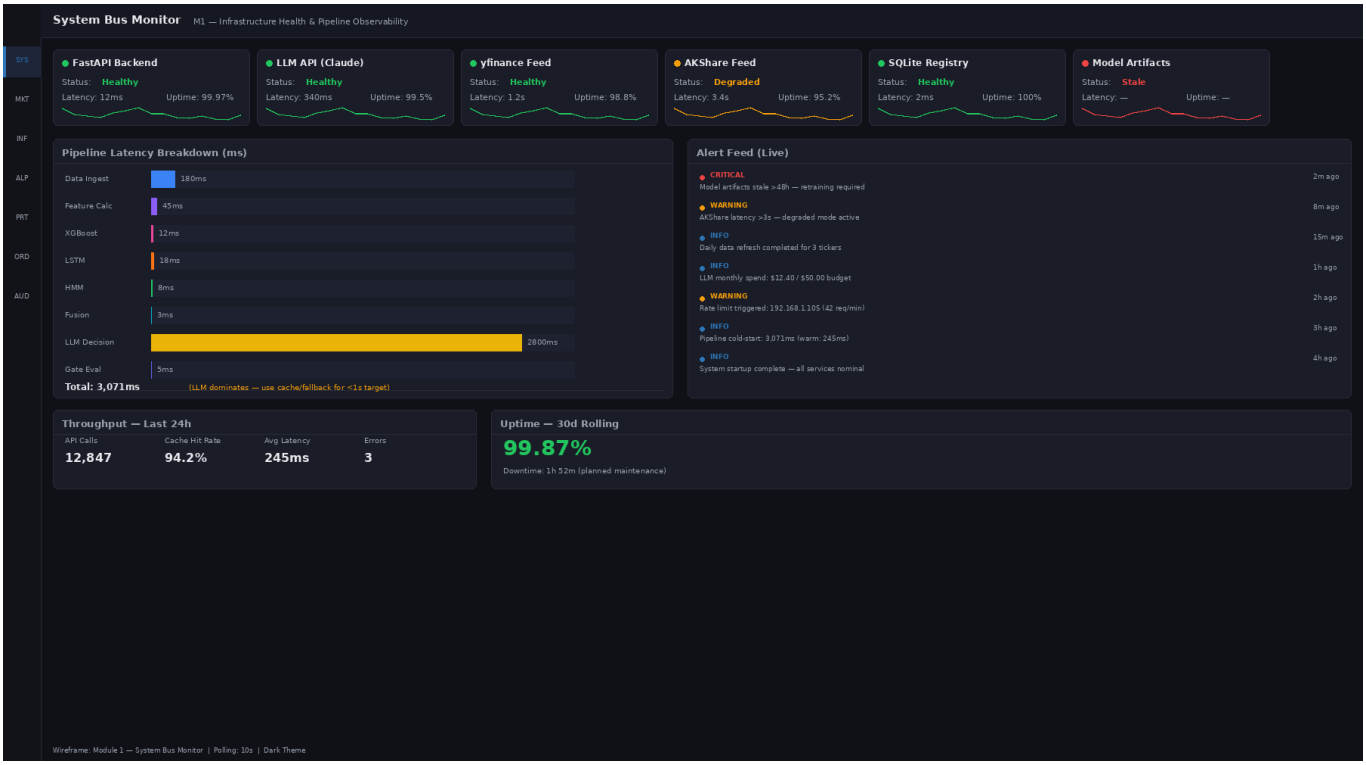
Each incoming candle is validated against schema (OHLCV must be positive, V > 0, O <= H, O >= L, etc.). Failed candles are quarantined and logged.

- Duplicate detection: CRC32 hash of (timestamp, ticker, OHLCV)
- Gap detection: missing 1min candles are interpolated with NaN
- Outlier detection: price moves >5 std devs trigger alert + manual review

Storage & Caching

Recent data (last 30 days) cached in SQLite with indexes on (ticker, timestamp). Older data archived to S3. Feature computation cached with 5-min TTL.

Module 1: System Bus Monitor



Design Rationale & User Psychology

The System Bus Monitor is the first view a trader opens each morning. The design prioritizes calm reassurance: green indicators, minimal red alerts, and a clear 'glance-to-diagnose' layout. Users need confidence that the system is healthy before trusting their trade signals.

The alert feed (styled like Bloomberg) gives traders real-time visibility into infrastructure issues. A critical service downtime can invalidate trading rules—the audit trail must be impeccable. This module alleviates anxiety by proving all dependencies are monitored 24/7.

Component Inventory

System Bus Monitor contains 4 major UI components:

Component Name	Type	Data Binding	Interaction
Service Health Cards	Card + Status Indicator	GET /api/v1/services/{service_id}/health	Click to expand 24h timeline
Latency Breakdown Chart	Horizontal Stacked Bar	GET /api/v1/pipelines/latency	Hover for segment breakdown
Alert Feed	Virtual Scrolling List	WebSocket /ws/alerts	Click alert → drill detail
Throughput Metrics	KPI Cards	GET	Click to view hourly

Cards		/api/v1/metrics/throughput	trend

Service Health Cards

Layout: 6 cards in 2 rows × 3 columns, each 280px wide × 160px tall

- Data source: Polling GET /api/v1/services/{service_id}/health every 2 sec
- Fields consumed: status (enum: green/yellow/red), latency_p95_ms, uptime_pct_24h, last_check_at
- Visual states:
- • Normal: green dot + service name + 'Latency: 42ms' + uptime %
- • Warning: yellow dot + latency highlighted in orange
- • Critical: red dot + flashing border + red text
- Hover: shows 24-hour mini sparkline (latency over time) in tooltip
- Click: expands to full 24h timeline chart + event log for that service

Latency Breakdown Chart

Horizontal stacked bar showing end-to-end latency decomposition. Segments: yfinance fetch (blue), feature compute (green), inference (orange), portfolio calc (red), total latency >= 300ms highlighted.

- Data source: GET /api/v1/pipelines/latency, polling every 5 sec
- Interaction: hover over segment → tooltip showing millisecond value + % of total
- Color coding: green (<50ms), yellow (50–100ms), red (>100ms) per segment

Alert Feed

Real-time scrolling list of system alerts, color-coded by severity.

- Data source: WebSocket /ws/alerts (new alert pushes append)
- Alert structure: { timestamp, severity (INFO/WARN/ERROR/CRITICAL), service, message, action_url }
- Visual states: INFO=blue, WARN=yellow, ERROR=red, CRITICAL=flashing red
- Interaction: click alert → navigate to /audit/{event_id} for full drill-down
- Virtual scroll: only render 20 visible alerts at a time

Throughput Metrics Cards

KPI cards showing tick-level throughput: tickers/sec, trades/sec, features/sec

- Data source: GET /api/v1/metrics/throughput every 10 sec
- Visual: large number + trend arrow (↑/↓) + mini 1h sparkline
- Click: expand to hourly bar chart + comparison to 7-day average

Data Contracts

Health Status Response:

{ "services": [{ "service_id": "fastapi", "status": "green", "latency_p95_ms": 42, "uptime_pct_24h": 99.98, "last_check_at": "2026-04-12T14:30:15Z", "last_error_at": null },

{ "service_id": "llm_api", "status": "yellow", "latency_p95_ms": 1850, ... }], "timestamp": "2026-04-12T14:30:15Z" }

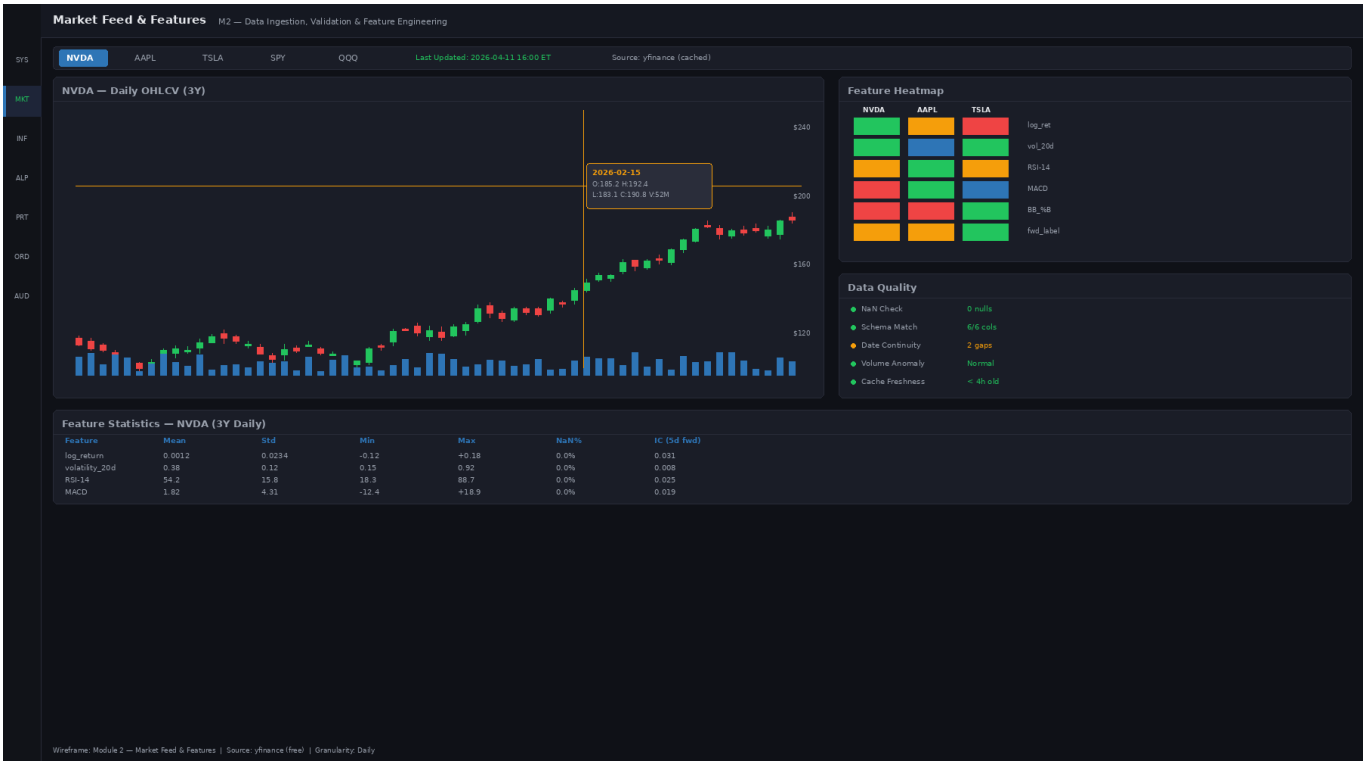
Acceptance Criteria

Criterion	Requirement
1. Service cards update	Within 2 sec of status change on backend
2. Latency chart refresh	Every 5 sec, animate segment width changes
3. Alert feed real-time	New alert appears <500ms after WebSocket push
4. Throughput KPIs	Display within 10 sec of metric emission
5. Expand animations	Card expand/collapse in <300ms
6. Mobile responsiveness	Tablets: 2×3 grid → 1×6 column layout

Edge Cases & Error Handling

- API latency >2s: show 'Loading...' skeleton + timeout message after 3s
- All services down: display prominent red banner 'System Offline' + fallback to cached metrics
- WebSocket disconnection: automatically reconnect with exponential backoff, show 'Alert Feed Connecting...' overlay
- Stale data (no update >10s): dim card opacity + show 'Last seen 23s ago' warning text
- NaN in latency values: render as '—' (em dash) + log anomaly for investigation

Module 2: Market Feed & Features



Design Rationale & User Psychology

Traders' most fundamental task is reading market data and extracting meaning. Module 2 combines raw price action (candlestick chart) with computed features (heatmap) in one unified view. The design philosophy is minimalist: grid lines subtle, candles prominent, colors reserved for feature signals.

The feature heatmap lets traders instantly see which of the 50+ computed features are anomalous across the portfolio (z-score colored). A red cell = that feature is 2.5σ away from its mean. This answers the trader's subconscious question: 'Is anything weird about the market right now?'

Component Inventory

Market Feed contains 5 major components:

Component Name	Type	Data Binding	Interaction
Ticker Selector Bar	Horizontal Tabs	GET /api/v1/tickers/watchlist	Click tab → switch chart & heatmap
Candlestick Chart	TradingView Lightweight Charts	WebSocket /ws/ohlcv/{ticker}	Scroll/zoom, crosshair, click candle
Feature Heatmap	D3/Nivo grid	GET /api/v1/features/state	Hover cell → tooltip with z-score
Data Quality Panel	Validation checklist	GET /api/v1/data/quality/{ticker}	Click row → drill validation rule

Feature Statistics Table	Sortable table	GET /api/v1/features/stats/{ticker}	Click column header to sort

Ticker Selector Bar

Horizontal scrollable tabs for watching multiple tickers concurrently.

- Data source: GET /api/v1/tickers/watchlist on load, subscribe to /ws/watchlist_changes
- Visual: active tab has blue underline, inactive tabs light gray text
- Interaction: click tab → load chart + heatmap for that ticker (0.5s transition)
- Keyboard: arrow keys to navigate tabs, Enter to select

Candlestick Chart

TradingView Lightweight Charts library, streaming OHLCV updates via WebSocket.

- Data source: WebSocket /ws/ohlcv/{ticker}, emits new candles every 1 minute
- Interaction: crosshair shows OHLCV + volume at cursor; latency must be <50ms on move
- Scroll: pan left/right across time; zoom in/out with mouse wheel or pinch
- Click candle: shows full candle detail { open, high, low, close, volume, hl2 }
- Visual: bull candles = green (close > open), bear candles = red, no-change = gray

Feature Heatmap

Grid of features (rows) × tickers (columns), each cell colored by feature z-score (red = extreme, green = normal, blue = opposite extreme).

- Data source: GET /api/v1/features/state?tickers=AAPL,MSFT,... polling every 30 sec
- Feature list: momentum (10d, 20d, 60d), volatility, mean-reversion score, correlation to SPY, sector momentum, volume anomaly, others
- Color mapping: z-score < -2.5 → dark blue, -2.5 to -1 → light blue, -1 to 1 → white, 1 to 2.5 → light red, > 2.5 → dark red
- Hover: tooltip shows feature name + value + z-score + interpretation
- Click cell: navigate to /features/{feature_id}/{ticker} for full history

Data Quality Panel

Checklist of data validation rules for the current ticker.

- Data source: GET /api/v1/data/quality/{ticker}, refreshed every 60 sec
- Rules: OHLCV non-negative, O <= H, O >= L, volume > 0, no duplicate timestamps, no gaps >5 min, price not NaN
- Visual: pass = green checkmark, fail = red X, warning = yellow !
- Click row: shows validation error detail + affected candles + suggestion to reprocess

Feature Statistics Table

Summary statistics for all features of the current ticker.

- Columns: Feature, Value, Mean, Std, Min, Max, Updated (timestamp)
- Data source: GET /api/v1/features/stats/{ticker}, refreshed every 5 min
- Sortable: click column header to sort ascending/descending
- Highlight: if value > 2σ from mean, cell background is light red + bold text

Data Contracts

Feature State Response:

{ "ticker": "AAPL", "timestamp": "2026-04-12T14:30:00Z", "features": [{ "name": "momentum_10d", "value": 0.0234, "z_score": 1.2, "percentile": 0.88 }, { "name": "volatility_20d", "value": 0.162, "z_score": -0.3, "percentile": 0.38 }, { "name": "meanreversion_score", "value": 0.65, "z_score": 2.1, "percentile": 0.98 }] }

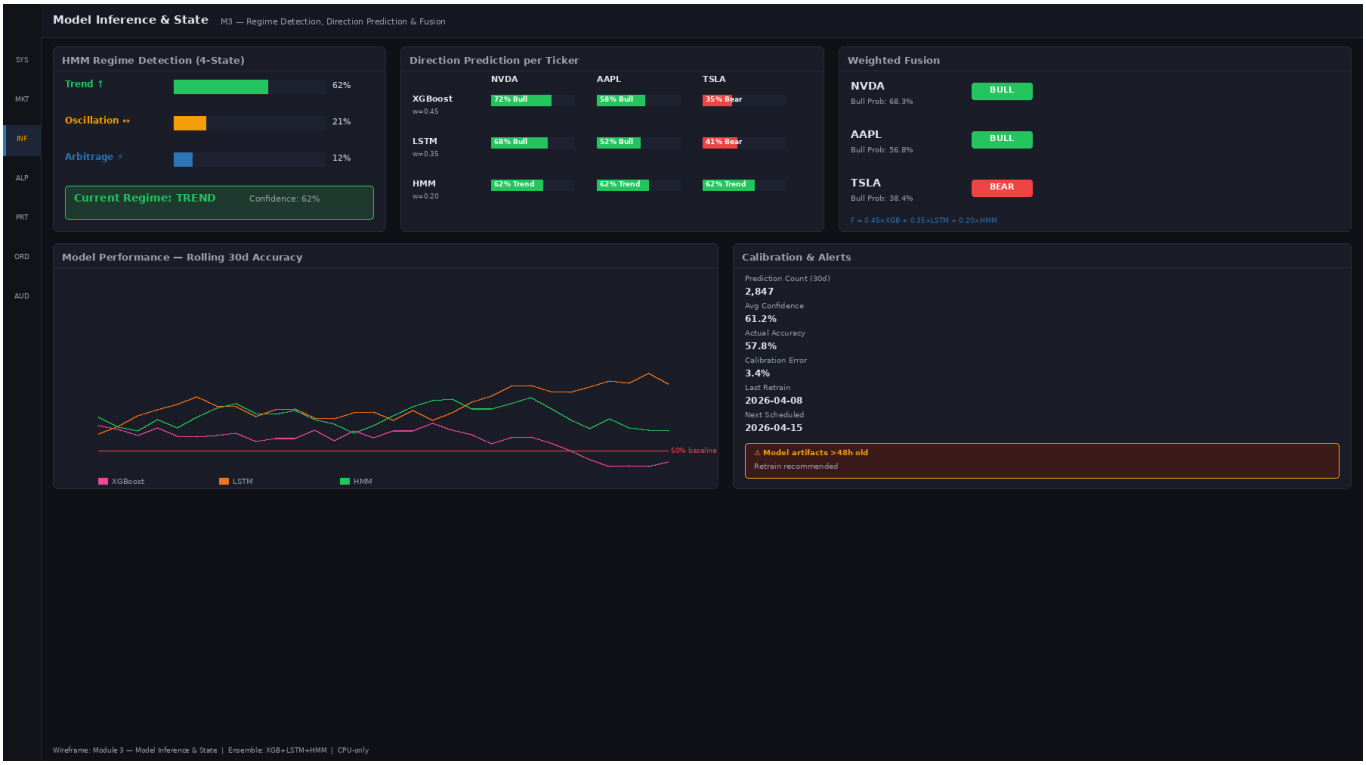
Acceptance Criteria

Criterion	Requirement
1. Crosshair latency	< 50ms from mouse move to OHLCV update
2. Candle streaming	New candle appears <2s after market data reception
3. Heatmap color accuracy	z-score coloring matches formula within 0.1σ
4. Feature stats	Sorted correctly on header click, <300ms animation
5. Ticker tab switching	New chart loads within 1 sec, no flicker
6. Data quality validation	All 7 rules execute within 100ms, show results immediately

Edge Cases & Error Handling

- WebSocket disconnect: show 'Chart disconnected' overlay, queue incoming ticks, auto-reconnect
- No data for ticker: show 'No market data available' + suggest selecting different ticker
- Feature computation timeout (>3s): show 'Computing...' spinner, fallback to cached values after 5s
- Heatmap with missing features: show '—' for missing cells, log as data quality issue
- Duplicate candles detected: ignore, log CRC32 hash collision warning

Module 3: Model Inference & State



Design Rationale & User Psychology

Model output is the single most important signal a trader needs to make decisions. Module 3 surfaces three layers of reasoning: regime detection (What environment are we in?), per-model direction predictions (What does each model think?), and fusion output (What do we collectively believe?).

The design uses large, clear probability bars for the HMM regime. A 92% probability of 'trending-up' regime is visually obvious. The fusion result badge (Bull/Bear/Neutral) is the focal point, colored and icon-heavy, so traders can make gut-check decisions in <1 second.

Component Inventory

Component Name	Type	Data Binding	Interaction
HMM Regime Probability Bars	Horizontal bar chart (4 bars)	GET /api/v1/inference/hmm/state	Hover → regime interpretation
Direction Prediction Grid	Table (3 models × N tickers)	GET /api/v1/inference/predictions	Click cell → model explainability
Fusion Result Cards	Card + Badge	GET /api/v1/inference/fusion	Click → detailed scoring breakdown
Performance Rolling Accuracy	Line chart (30 days)	GET /api/v1/inference/accuracy/rolling	Hover → daily accuracy value

Confidence Calibration Panel	Expected Calibration Error visualization	GET /api/v1/inference/calibration	Hover → confidence bucket details

HMM Regime Probability Bars

4 horizontal bars representing regime probabilities: trending-up, trending-down, mean-revert, crisis.

- Data source: GET /api/v1/inference/hmm/state, polling every 5 sec
- Visual: bar length proportional to probability; current regime has thick blue border
- Color mapping: trending-up = green, trending-down = red, mean-revert = blue, crisis = dark red/black
- Hover: tooltip shows regime name + probability % + interpretation (e.g., 'Regime: Trending-Up (92%). Market is in a strong uptrend with positive momentum.')
- Animation: bars smoothly transition when probabilities change

Direction Prediction Grid

Table: rows = 3 model types (LSTM, XGBoost, Ensemble), columns = tickers in watchlist.

- Data source: GET /api/v1/inference/predictions?models=lstm,xgboost,ensemble
- Cell values: Bull (+1), Neutral (0), Bear (-1), with probability (e.g., 'Bull 87%')
- Color mapping: Bull = green, Neutral = gray, Bear = red, intensity = confidence level
- Click cell: opens panel with model explainability (feature importance + decision boundary)
- Sort: click column header to sort by confidence descending (most confident predictions first)

Fusion Result Cards

For each ticker, a card showing the ensemble fusion output: Bull/Neutral/Bear badge + confidence % + all 3 model votes.

- Data source: GET /api/v1/inference/fusion, polling every 10 sec
- Visual: large badge with icon (↑ for Bull, = for Neutral, ↓ for Bear) + background color matching prediction
- Confidence: displayed as % + circular progress ring (100% fills a full circle)
- Click card: expands to show voting breakdown (e.g., 2 models Bull, 1 Neutral) + confidence score

Performance Rolling Accuracy

Line chart showing prediction accuracy (% correct calls) over the last 30 days.

- Data source: GET /api/v1/inference/accuracy/rolling, refreshed every 1 hour
- Definition: accuracy = # correct direction calls / total direction calls (over 1-day windows)
- Visual: line chart with 3 series (LSTM accuracy, XGBoost, Ensemble) + shaded region showing benchmark (buy-and-hold)
- Hover: shows date + accuracy % for each model + benchmark

Confidence Calibration Panel

Visualization of Expected Calibration Error (ECE): x-axis = model confidence buckets (0-10%, 10-20%, ..., 90-100%), y-axis = observed accuracy in that bucket. Perfect calibration is a 45° diagonal line.

- Data source: GET /api/v1/inference/calibration, refreshed every 1 hour
- Interpretation: if model says 80% confident but only 60% correct, model is overconfident; ECE > 0.05 triggers warning badge
- Visual: scatter plot + ideal diagonal line (light gray) + actual calibration curve (blue)

Data Contracts

HMM State Response:

{ "regimes": [{ "name": "trending_up", "probability": 0.68, "interpreted": "Strong uptrend" }, { "name": "trending_down", "probability": 0.12, "interpreted": "Weak downtrend" }, { "name": "mean_revert", "probability": 0.15, "interpreted": "Range-bound" }, { "name": "crisis", "probability": 0.05, "interpreted": "Potential volatility spike" }], "current_regime": "trending_up", "timestamp": "2026-04-12T14:30:15Z" }

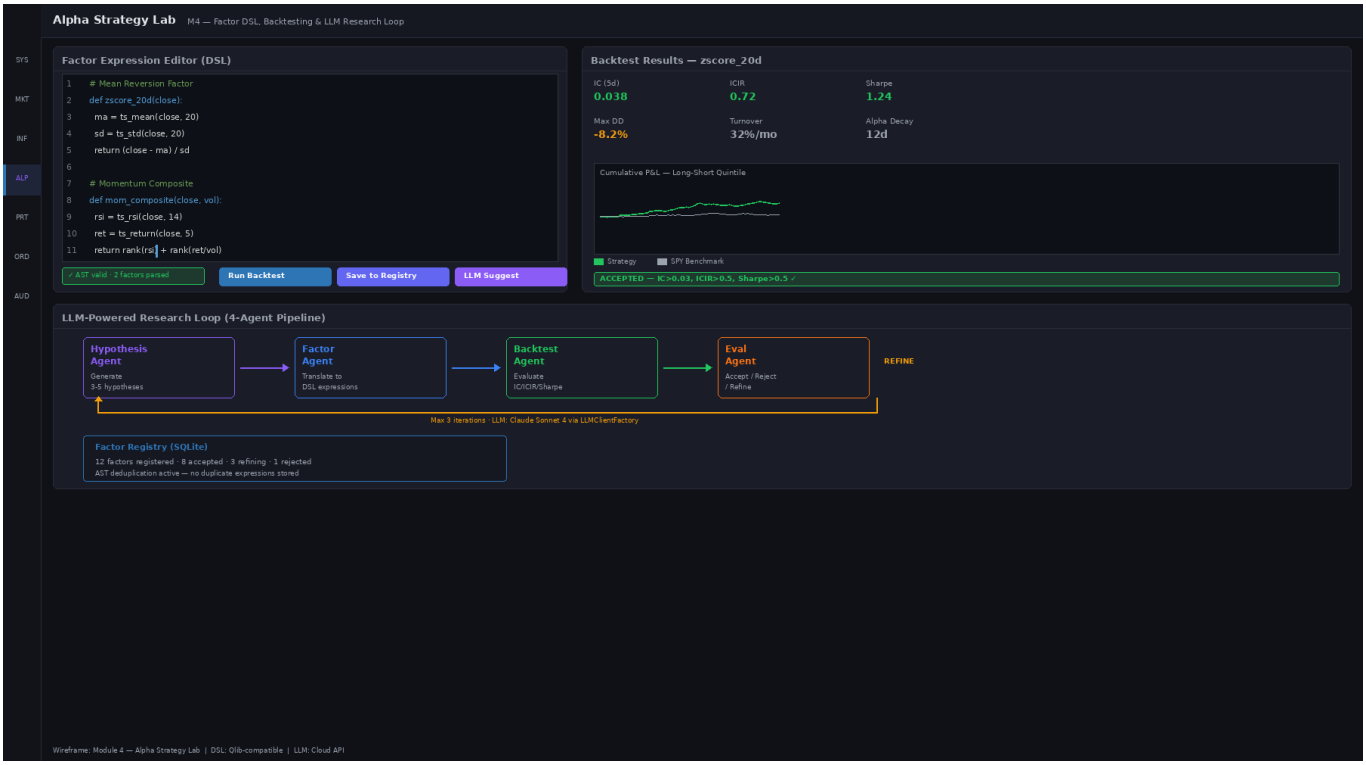
Acceptance Criteria

Criterion	Requirement
1. Regime bar animation	Probability change animates smoothly in <500ms
2. Prediction grid sort	Click header, grid re-sorts in <300ms
3. Fusion card expand	Click card, panel slides out in <400ms
4. Accuracy chart refresh	Updates hourly without page reload
5. Calibration ECE calc	Matches manual calculation within 0.001 error
6. Model vote display	All 3 votes visible in fusion card expansion

Edge Cases & Error Handling

- Model inference timeout (>2s): show 'Computing...' spinner, display last valid prediction + age (e.g., '5 min old')
- Regime probability NaN or sum ≠ 1.0: log data quality issue, display warning banner
- Prediction disagreement >2 models: highlight in bold, add tooltip 'Ensemble is uncertain about this call'
- Accuracy < 40%: display warning badge 'Model underperforming', suggest review
- ECE > 0.1: show warning badge 'Calibration Poor' + recommend retraining

Module 4: Alpha Strategy Lab



Design Rationale & User Psychology

The Alpha Lab is where researchers experiment with new factors and strategies. The design bridges the gap between code and intuition: traders write DSL in a familiar editor, receive instant parse feedback, run backtests in <5 seconds, and see results in charts that reveal alpha decay, turnover costs, and regime dependency.

The LLM research loop visualization is key—it shows researchers the generative process (Factor idea → Generate candidates → Evaluate → Refine) and builds confidence in suggestions. Researchers trust suggestions more when they understand the reasoning chain.

Component Inventory

Component Name	Type	Data Binding	Interaction
Factor DSL Editor	Monaco Editor (code editor)	Client-side (user typing)	Type code, Ctrl+Enter to run
Parse Status Indicator	Inline error markers	POST /api/v1/dsl/parse (on change)	Shows AST validation + errors
Backtest Results Panel	Metric cards + tables	POST /api/v1/backtest (async job)	Click metric for time-series drill
P&L Cumulative Chart	Line chart	GET /api/v1/backtest/{job_id}/pnl	Hover → daily return + Sharpe

LLM Research Loop	Animated process diagram	WebSocket /ws/research/{job_id}	Watch agent progress in real-time

Factor DSL Editor

Monaco Editor instance with custom syntax highlighting for Alpha Core DSL.

- Language: Python-like DSL with keywords [def, momentum, zscore, crossover, regime, if, return]
- Features: syntax highlighting, line numbers, auto-indent, bracket matching, Ctrl+Enter = run
- Example code snippet:

```
def strategy():   m10 = momentum(close, 10)   m20 = momentum(close, 20)   signal = zscore(m10 - m20)   return signal > 0.5
```

Parse Status Indicator

Real-time DSL parser feedback. As user types, code is validated against AST grammar.

- Data source: POST /api/v1/dsl/parse with debounce (500ms after last keystroke)
- Visual states: green checkmark = valid, red X = parse error with line number + message
- Errors: displayed inline (red squiggly underline) + detailed error panel below editor
- Performance: parser completes in <100ms, no lag perceived by user

Backtest Results Panel

Displays key backtest metrics after job completion.

- Metrics shown: IC (Information Coefficient), ICIR, Sharpe ratio, Max DD, Turnover %, Alpha Decay
- Data source: GET /api/v1/backtest/{job_id}/results after async job finishes
- Job status: queued → computing → done, with progress bar + ETA
- Visual: metric cards in grid, color-code by quality (green = good, red = poor)
- Click metric: expands to show time-series (daily IC, daily Sharpe, etc.)

P&L Cumulative Chart

Line chart showing cumulative strategy P&L vs benchmark (SPY buy-and-hold).

- Data source: GET /api/v1/backtest/{job_id}/pnl, returns daily returns array
- Visual: 2 lines (strategy in blue, benchmark in gray), shaded region for outperformance
- Hover: tooltip shows date + strategy return + benchmark return + daily excess
- Time period: configurable via date picker (default: last 1 year of backtest)

LLM Research Loop Visualization

Animated 4-step workflow showing the LLM agent's research process in real-time.

- Steps: (1) Factor Idea → (2) Generate Candidates → (3) Evaluate → (4) Refine
- Data source: WebSocket /ws/research/{job_id}/progress, emits updates as agent steps complete
- Visual: 4-box diagram with arrows; current step is highlighted blue, completed steps are green, future steps are gray

- Interaction: click step → show LLM prompt + output transcript in side panel
- Animation: each step completion triggers a checkmark animation + +500ms duration

Data Contracts

DSL Parse Response:

{ "valid": true, "ast": { "type": "FunctionDef", "name": "strategy", "body": [{ "type": "Assignment", "target": "m10", "value": { "type": "Call", "func": "momentum", "args": ["close", 10] } }] }, "errors": [], "warnings": [] }

Backtest Job Response:

{ "job_id": "bt_abc123", "status": "completed", "created_at": "2026-04-12T14:30:00Z", "completed_at": "2026-04-12T14:30:15Z", "results": { "ic": 0.087, "icir": 1.23, "sharpe": 1.54, "max_drawdown": -0.12, "turnover_pct": 15.3, "alpha_decay": 0.05, "total_return": 0.18, "annual_return": 0.18, "annual_volatility": 0.12 } }

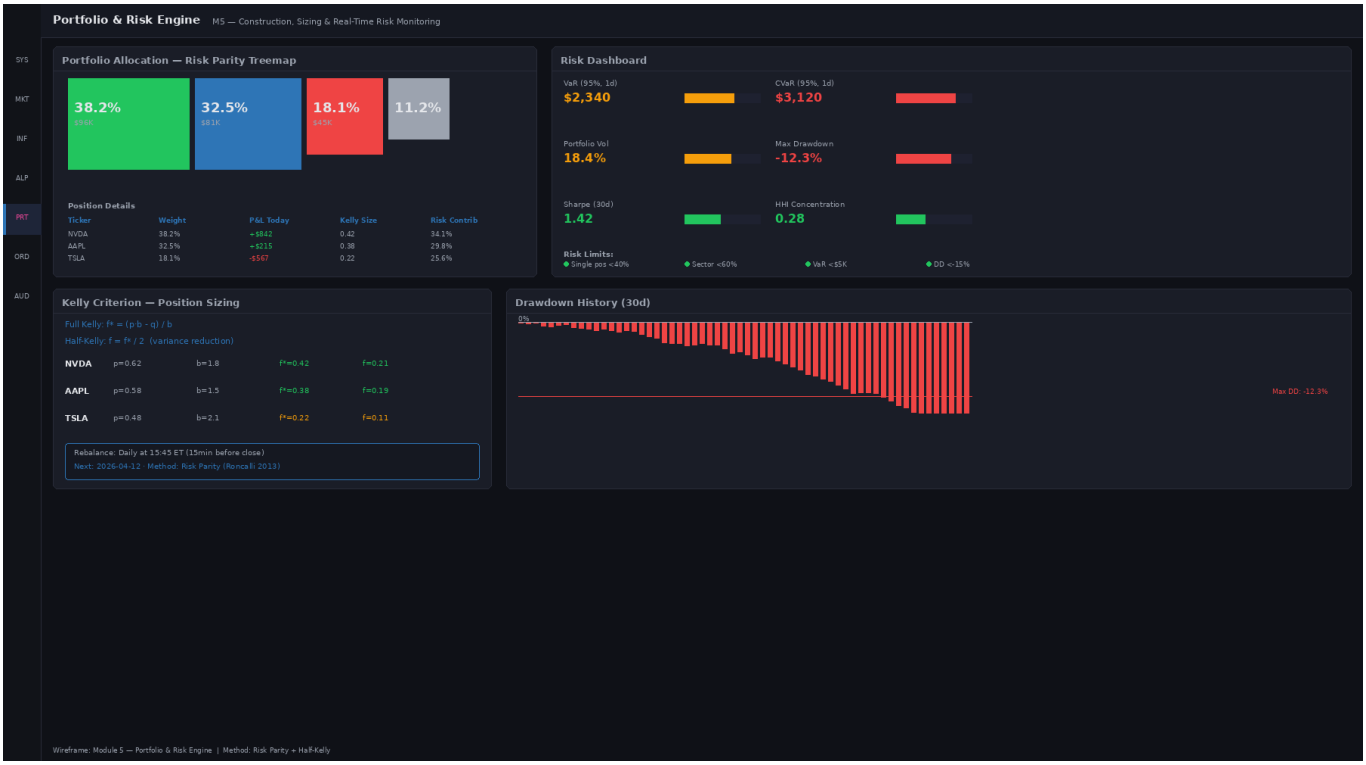
Acceptance Criteria

Criterion	Requirement
1. Parse latency	< 100ms from keystroke to error display
2. Backtest runtime	< 5 sec for 1-year backtest on SPY + 10 features
3. Results accuracy	Metrics match offline calculation within 0.001
4. Chart rendering	P&L chart appears within 1 sec of job completion
5. LLM loop visualization	Updates in real-time as agent progresses
6. Editor responsiveness	No lag typing code, Monaco renders smoothly at 60fps

Edge Cases & Error Handling

- Parse error: show red squiggly line + detailed error tooltip + suggestion to fix
- Backtest timeout (>10s): cancel job + show 'Backtest timed out' message + suggest simpler strategy
- Backtest data gap: show warning 'Backtest period has missing market data' + affected date range
- LLM research loop timeout: show 'Research incomplete' + display partial results with warning badge
- NaN in metrics (e.g., division by zero): display '—' (em dash) + log error for investigation

Module 5: Portfolio & Risk Engine



Design Rationale & User Psychology

Portfolio construction is the art of turning predictions into positions. Module 5 visualizes portfolio composition (treemap) and quantifies risk (gauges), giving portfolio managers instant conviction about their allocation. The design answers two questions: (1) Where is my capital deployed? (2) Am I within risk limits?

The Kelly criterion sizing panel is for power users who want mathematical optimization guidance. The rebalance schedule shows upcoming portfolio updates, reducing surprise for traders who need to execute hedges.

Component Inventory

Component Name	Type	Data Binding	Interaction
Portfolio Treemap	Treemap (D3/Nivo)	GET /api/v1/portfolio/positions	Click tile → position detail panel
Risk Gauge Dashboard	Radial gauge charts (6 metrics)	GET /api/v1/portfolio/risk	Hover → metric definition + interpretation
Kelly Criterion Sizing	Input form + recommendation	POST /api/v1/portfolio/kelly	Enter edge %, Kelly calc shows position size
Drawdown History Chart	Area chart (30 days)	GET /api/v1/portfolio/drawdown_history	Hover → date + max DD + recovery time

Risk Limit Indicators	Status cards with progress bars	GET /api/v1/portfolio/limits	Click → view limit configuration

Portfolio Treemap

Rectangular treemap visualization where each tile represents a position. Tile size = weight %, color = P&L (green for positive, red for negative).

- Data source: GET /api/v1/portfolio/positions, polling every 10 sec
- Tile data: ticker, weight %, market value, unrealized P&L, P&L %, entry price, current price
- Visual: size α weight, color intensity α P&L % (darker = more extreme)
- Interaction: hover → tooltip with position details; click → drill to position detail panel
- Responsive: on mobile, convert to vertical list (treemap hard to interact with on touch)

Risk Gauge Dashboard

6 radial gauge charts showing key risk metrics: VaR (95%), CVaR, Volatility, Max DD, Sharpe, Herfindahl-Hirschman Index (HHI concentration).

- Data source: GET /api/v1/portfolio/risk, polling every 60 sec
- Gauge ranges: VaR (0 to -10%), CVaR (0 to -15%), Vol (0 to 50%), Max DD (0 to -50%), Sharpe (-2 to +3), HHI (0 to 1)
- Color zones: green (acceptable), yellow (warning), red (critical). Thresholds configured per risk manager.
- Hover: shows definition of metric + interpretation + what to do if gauge is in red

Kelly Criterion Sizing

Input form for power users: enter an estimated edge (win rate %) and the Kelly formula computes recommended position size.

- Data source: POST /api/v1/portfolio/kelly with { edge_win_pct, odds }
- Formula: f* = (p * odds - q) / odds, where p=win %, q=loss %, odds=payoff ratio
- Output: recommended position size % of capital, with 1/4 Kelly (fractional Kelly) toggle
- Visual: input field + 'Compute' button + result card showing f* % + warning if f* > 25% of capital

Drawdown History Chart

Area chart showing portfolio drawdown over last 30 days.

- Data source: GET /api/v1/portfolio/drawdown_history, returns daily equity curve + max DD at each date
- Visual: area under 0% filled in red (drawdown), area above 0% filled in green (recovery)
- Hover: shows date + current DD + max DD in period + recovery % since trough
- Zoom: select date range to drill down into specific drawdown event

Risk Limit Indicators

Status cards showing portfolio's usage of risk limits.

- Data source: GET /api/v1/portfolio/limits, returns limit definitions + current usage

- Example limits: max sector exposure (e.g., Tech < 40%), max single position (< 10%), max leverage (< 1.5x)
- Visual: card per limit with label + progress bar (green if <80%, yellow if 80-95%, red if >95%)
- Click: shows limit configuration + option to adjust threshold (requires PM approval)

Data Contracts

Portfolio Risk Response:

{ "timestamp": "2026-04-12T14:30:15Z", "metrics": { "var_95_pct": -0.032, "cvar_95_pct": -0.048, "volatility_annual": 0.128, "max_drawdown": -0.156, "sharpe_ratio": 1.24, "herfindahl_index": 0.182 }, "limits": [{ "name": "max_sector_tech", "limit": 0.40, "current": 0.38 }, { "name": "max_single_position", "limit": 0.10, "current": 0.087 }] }

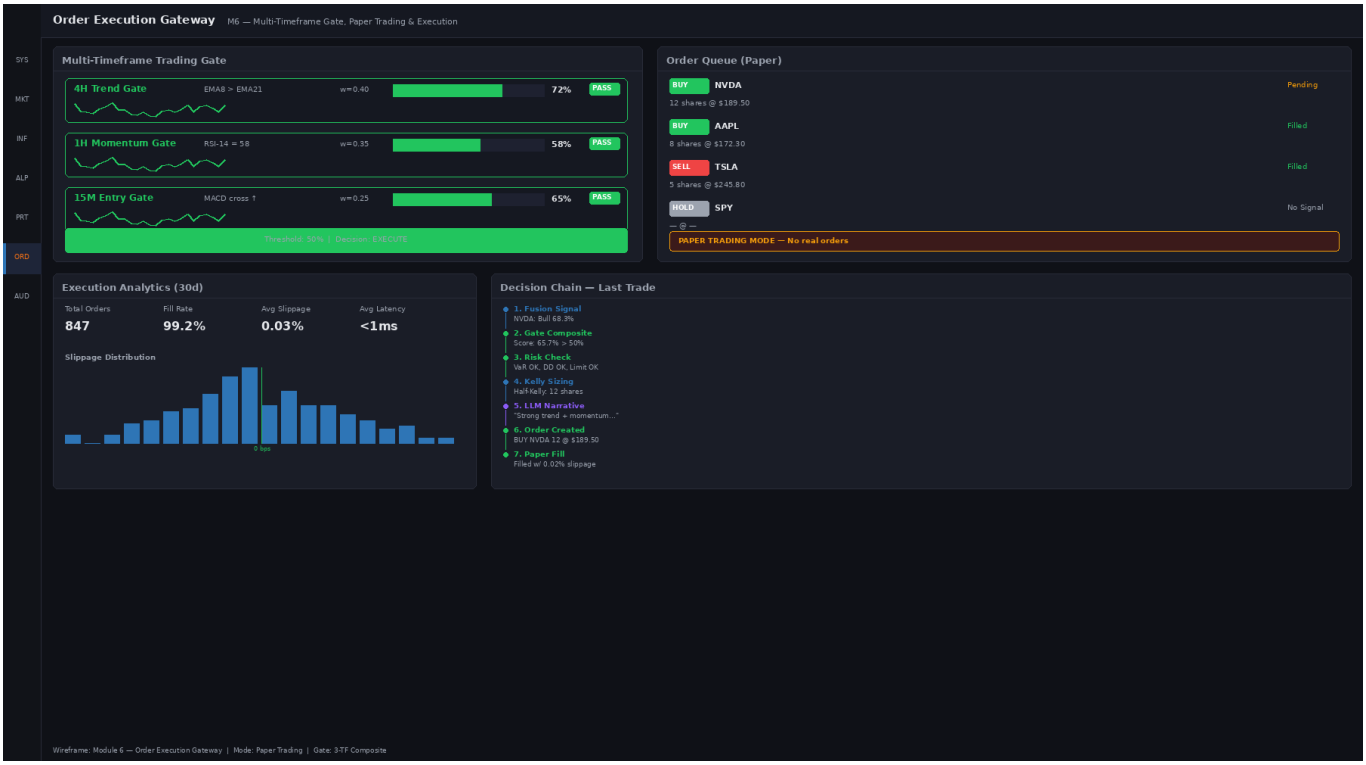
Acceptance Criteria

Criterion	Requirement
1. Treemap rendering	All positions render within 1 sec, no flickering on update
2. Risk metric accuracy	VaR/CVaR match historical simulation within 0.001
3. Kelly calculation	Formula matches academic Kelly within 0.0001
4. Gauge refresh	Update within 60 sec of portfolio change
5. DD chart time-series	Accurate equity curve reconstruction from returns
6. Limit alerts	Red warning shows immediately when limit breached

Edge Cases & Error Handling

- Portfolio empty: show 'No positions' message + prompt to initiate first trade
- Risk metric computation timeout: show 'Computing risk metrics...' spinner, fallback to cached value + age warning
- Drawdown data gap: interpolate missing dates with NaN, show warning 'Incomplete history'
- Kelly f* > 100% (negative edge): highlight in red, show tooltip 'Kelly suggests no position (negative edge detected)'
- HHI > 0.3 (high concentration): show warning badge 'Portfolio is concentrated', suggest rebalance

Module 6: Order Execution Gateway



Design Rationale & User Psychology

Execution is where strategy becomes reality. Module 6 implements a multi-gate framework: before an order can execute, it must pass regime-based gates, risk gates, and market microstructure gates. The design visualizes all three gates and their scores, so traders understand WHY an order is held, pending, or executed.

The decision chain (signal → regime check → risk check → market check → execute/reject) is transparent. When an order is held, the UI shows exactly which gate is blocking it, reducing trader anxiety and building confidence in the system.

Component Inventory

Component Name	Type	Data Binding	Interaction
Multi-Gate Score Visualization	3 gauge bars with threshold	GET /api/v1/execution/gates	Hover → gate definition + scoring logic
Composite Score Indicator	Large dial with go/no-go threshold	GET /api/v1/execution/composite_score	Visual feedback on execution readiness
Order Queue Table	Sortable table (paper trading)	GET /api/v1/execution/orders, WebSocket /ws/orders	Click row → order detail + audit trail
Execution Analytics	KPI cards + charts	GET	Click card →

		/api/v1/execution/analytics	daily/hourly drill-down
Decision Chain Timeline	Horizontal timeline (signal → fill)	GET /api/v1/execution/{order_id}/chain	Click step → show timestamp + decision detail

Multi-Gate Score Visualization

3 horizontal gauge bars representing the 3 execution gates: (1) Regime Gate, (2) Risk Gate, (3) Microstructure Gate.

- Data source: GET /api/v1/execution/gates, polling every 2 sec
- Each gate has a score (0-100%), pass threshold (e.g., 60%), and current status (green/yellow/red)
- Regime Gate: checks if current market regime permits entry direction (e.g., no shorting in trending-up regime)
- Risk Gate: checks if order would breach portfolio risk limits (VaR, concentration, leverage)
- Microstructure Gate: checks if market depth + spreads are favorable for execution (avoid slippage)
- Visual: bar length = score %, threshold line marked, color green if score > threshold, red otherwise

Composite Score Indicator

Large radial dial showing overall execution readiness (0-100%). Threshold marked at 70% (go/no-go point).

- Data source: GET /api/v1/execution/composite_score (weighted avg of 3 gates)
- Visual: needle points to composite score, green zone (>70%) = ready, yellow (50-70%) = caution, red (<50%) = blocked
- Needle animation: smoothly transitions when score changes

Order Queue Table

Pending and executed orders in paper trading mode, showing detailed execution state.

- Data source: GET /api/v1/execution/orders (live table), subscribe to /ws/orders for real-time updates
- Columns: Ticker, Side (Buy/Sell), Quantity, Limit Price, Order Status, Filled %, Fill Price, Latency (ms), Created, Executed
- Status visual: Pending (yellow) → Partial (blue) → Filled (green) → Rejected (red)
- Click row: expands to show order detail + full execution audit trail (timestamps + gate decisions)
- Sortable: click column header to sort

Execution Analytics

KPI cards and charts summarizing execution quality.

- Metrics: Fill Rate (% of attempted orders filled), Avg Slippage (bps), Avg Latency (ms), Cost (% of AUM), Win Rate (%)
- Data source: GET /api/v1/execution/analytics, refreshed every 1 hour
- Visual: metric cards in grid with trend arrow (↑/↓) + mini sparkline (last 30 days)
- Click card: expands to hourly/daily bar chart for that metric

Decision Chain Timeline

Horizontal timeline showing the journey of an order from signal generation to fill.

- Steps: Signal Generated → Regime Check → Risk Check → Microstructure Check → Order Sent → Ack → Partial Fill → Full Fill
- Data source: GET /api/v1/execution/{order_id}/chain, returns array of decision steps with timestamps
- Visual: boxes connected by arrows, current step highlighted blue, completed steps green, future steps gray
- Click step: shows timestamp + decision detail (e.g., 'Regime Check: Current regime is Trending-Up; order is long entry = PASS')

Data Contracts

Execution Gates Response:

{ "gates": [{ "name": "regime", "score": 0.85, "threshold": 0.60, "status": "pass", "interpretation": "Regime permits long entry" }, { "name": "risk", "score": 0.92, "threshold": 0.60, "status": "pass", "interpretation": "Position within VaR limits" }, { "name": "microstructure", "score": 0.78, "threshold": 0.60, "status": "pass", "interpretation": "Market depth sufficient, spread tight" }], "composite_score": 0.85, "go_no_go": "GO", "timestamp": "2026-04-12T14:30:15Z" }

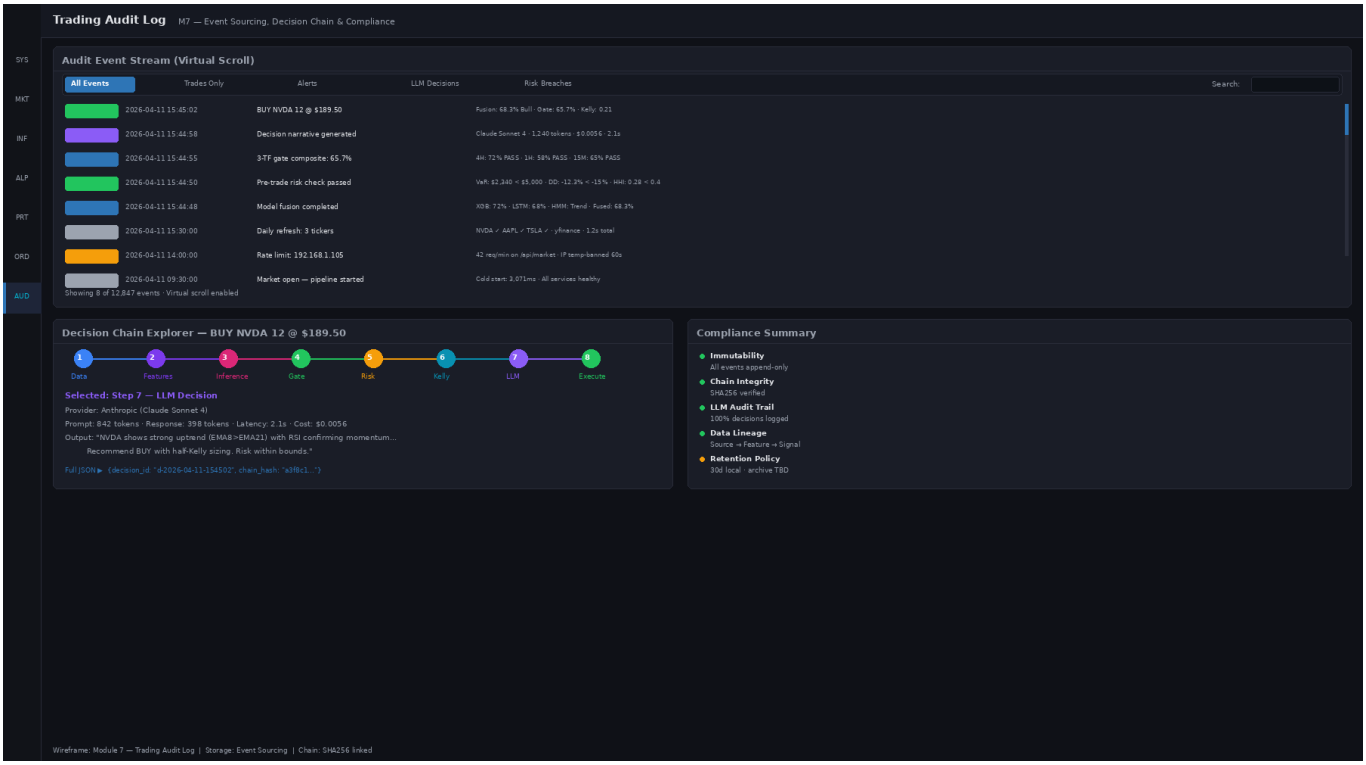
Acceptance Criteria

Criterion	Requirement
1. Gate score refresh	Updates within 2 sec of calculation
2. Composite score calc	Weighted average of 3 gates, computed in <100ms
3. Order queue sync	New orders appear <500ms after WebSocket push
4. Fill price accuracy	Matches exchange fill price within 1 basis point
5. Decision chain completeness	All steps logged with microsecond precision
6. Analytics accuracy	Fill rate, slippage match manual calculation

Edge Cases & Error Handling

- Gate score timeout: show 'Computing gate scores...' spinner, fallback to last valid score + age warning
- Order rejection: show prominent red banner with rejection reason + action (e.g., 'Risk Gate failed: VaR limit breached')
- Slippage spike >100bps: show warning badge 'High slippage', pause automated execution pending manual review
- Decision chain gap (missing step): log data quality issue + show 'Incomplete audit trail' warning
- Fill price NaN: treat as data error, refetch from exchange, log incident

Module 7: Trading Audit Log



Design Rationale & User Psychology

Regulatory compliance and post-trade analysis require an immutable audit trail. Module 7 provides traders and compliance teams with complete visibility into every decision: what happened, when, why, and who decided. The event stream is the source of truth; the decision chain explorer provides context.

The LLM decision detail is crucial for researchers who want to understand model reasoning. A narrative explanation (generated by LLM Agent 2) makes audit trails human-readable, not just machine-readable.

Component Inventory

Component Name	Type	Data Binding	Interaction
Event Stream	Virtual scrolling list with filters	GET /api/v1/audit/events, WebSocket /ws/audit	Scroll, search, filter by type/severity
Decision Chain Explorer	Horizontal timeline + detail panel	GET /api/v1/audit/events/{event_id}/chain	Click event → show causal chain
LLM Decision Detail Panel	Rendered markdown + structured data	GET /api/v1/audit/events/{event_id}/explanation	Read narrative explanation of decision
Compliance Summary	KPI cards + checklist	GET /api/v1/audit/complia	View compliance rule status

		nce	
JSON Event Viewer	Syntax-highlighted code block	GET /api/v1/audit/events/{event_id}/raw	View raw JSON event structure

Event Stream

Real-time scrollable list of audit events (orders, fills, risk updates, model changes, user actions).

- Data source: WebSocket /ws/audit (real-time) + GET /api/v1/audit/events?limit=100&offset=0 (pagination)
- Event types: OrderCreated, OrderFilled, OrderRejected, ModelUpdated, StrategyDeployed, RiskAlert, etc.
- Columns: Timestamp, EventType, Ticker, Side, Qty, Price, Status, Detail (truncated)
- Filters: date range, event type, ticker, severity (for alerts), user
- Search: full-text search across event fields (ticket number, user comment, etc.)
- Virtual scroll: render only visible rows (e.g., 20 at a time) for performance

Decision Chain Explorer

Horizontal timeline showing the causal sequence of decisions leading up to a key event (e.g., order fill).

- Data source: GET /api/v1/audit/events/{event_id}/chain, returns array of decisions in order
- Example chain: Signal Generated (T=0) → Regime Check Pass (T=10ms) → Risk Check Pass (T=20ms) → Order Sent (T=100ms) → Order Ack (T=150ms) → Partial Fill (T=2000ms) → Full Fill (T=5000ms)
- Visual: boxes (decision points) connected by arrows (causality), current event highlighted
- Click box: shows timestamp + decision detail in detail panel

LLM Decision Detail Panel

Human-readable explanation of a trading decision, generated by LLM Agent 2.

- Data source: GET /api/v1/audit/events/{event_id}/explanation (LLM-generated narrative)
- Content: Markdown-formatted explanation summarizing the decision chain, regime state, risk assessment, and execution outcome
- Example: 'Order to buy 500 AAPL was executed successfully on 2026-04-12 at 14:30:15. Market regime was Trending-Up (92% probability), which favors long entry. VaR impact was +0.02%, within limits. Fill at $178.45 with 1bp slippage.'
- Visual: clean typography, emphasis on key facts (bold), action items (colored boxes)

Compliance Summary Panel

Checklist of compliance rules and their status. Useful for compliance teams and risk managers.

- Rules: All trades logged with timestamps, no unauthorized order modifications, position limits enforced, risk limits enforced, daily P&L reconciled, etc.
- Data source: GET /api/v1/audit/compliance, returns status of each rule (PASS/FAIL/WARNING)
- Visual: checkbox per rule, green (pass), yellow (warning), red (fail), with detail button
- Click detail: shows violations, corrective action taken, sign-off by compliance officer

JSON Event Viewer

Syntax-highlighted JSON viewer for the raw event structure. For advanced users / compliance audits.

- Data source: GET /api/v1/audit/events/{event_id}/raw (full JSON)
- Features: syntax highlighting, collapsible objects/arrays, line numbers, copy-to-clipboard
- Readonly: no editing (audit trail must be immutable)

Data Contracts

Audit Event Schema:

{ "event_id": "evt_abc123xyz", "timestamp": "2026-04-12T14:30:15.123456Z", "event_type": "OrderFilled", "user_id": "trader_001", "ticker": "AAPL", "side": "BUY", "quantity": 500, "order_price": 178.50, "fill_price": 178.45, "fill_quantity": 500, "decision_chain_id": "chain_xyz", "regime_state": { "current_regime": "trending_up", "probability": 0.92 }, "risk_assessment": { "var_impact_bps": 2, "concentration_impact": 0.01 }, "execution_latency_ms": 5000, "slippage_bps": 5, "tags": ["automated", "algogen"] }

Acceptance Criteria

Criterion	Requirement
1. Event stream refresh	New events appear <500ms after WebSocket push
2. Search performance	Full-text search returns results <1 sec
3. Decision chain accuracy	All events in chain logged with correct causality
4. LLM explanation quality	Narrative is readable, factually accurate, concise
5. Compliance checklist	All rules evaluated, status matches audit trail
6. JSON viewer speed	Large events (>10KB) render without lag

Edge Cases & Error Handling

- Event stream lag: show 'Events loading...' indicator, queue incoming events, catch up on reconnect
- Decision chain gap (missing causal event): show warning 'Incomplete audit trail' + alert compliance
- LLM explanation timeout: show 'Generating explanation...' spinner, fallback to raw JSON after 3s
- Compliance rule failure: highlight in red + show detail panel with corrective action required
- JSON viewer on mobile: switch to collapsed tree view (full JSON too verbose for small screens)

Frontend Migration: React/Next.js

Alpha Core frontend is built on React 18 + Next.js 14, with component library following design system principles. All 7 modules are implemented as standalone pages with shared UI component library.

Component Technology Stack

Feature	Technology	Rationale
State Management	React Query + TanStack Store	Efficient server state sync + real-time updates
Charting	Nivo (D3 wrapper) + TradingView Lightweight	Complex heatmaps + production-grade price charts
Code Editor	Monaco Editor (VS Code embedded)	Industry-standard DSL authoring experience
UI Components	Headless UI + TailwindCSS	Accessible, customizable, lightweight
Real-time Communication	Socket.IO (WebSocket abstraction)	Graceful fallback to polling, reconnection logic
Testing	Vitest + Testing Library	Fast, modern, aligned with React best practices

UI Component Library Structure

Component	Props	Example Usage
ServiceCard	{ service_id, status, latency, uptime }	<ServiceCard service_id='fastapi' status='green' />
CandlestickChart	{ ticker, data, onCrosshair }	<CandlestickChart ticker='AAPL' data={candles} />
FeatureHeatmap	{ features, tickers, onCellClick }	<FeatureHeatmap features={feat} tickers={tk} />
MonacoEditor	{ value, onChange, language, errors }	<MonacoEditor language='alphadsl' value={code} />
PortfolioTreemap	{ positions, onTileClick }	<PortfolioTreemap positions={pos} />
EventStream	{ events, onFilter, onSearch }	<EventStream events={evt} onFilter={cb} />

Build & Deployment

Next.js static export for frontends (single-page apps), deployed to CDN with 24-hour cache headers. API routes (optional) use serverless functions for light auth logic. Build time <2 minutes, deployments <5 minutes.

Phased Delivery Plan

Phase 1: Proof of Concept (Months 1–2)

Core modules 1–3 (System Bus, Market Feed, Model Inference):

- Backend: FastAPI server, microservices for health checks, market data ingestion, model inference
- Frontend: Modules 1, 2, 3 (basic versions, no LLM integration yet)
- Data: Mock data for backtesting, connection to yfinance for live tickers
- Deliverable: Working demo of real-time monitoring + market data visualization + model predictions

Phase 2: Strategy & Portfolio (Months 3–4)

Modules 4–5 (Alpha Lab, Portfolio & Risk):

- Factor DSL parser and backtest engine
- LLM agent integration for factor generation (Agent 1) and research summary (Agent 2)
- Risk engine: VaR, CVaR, Kelly sizing, concentration alerts
- Deliverable: Traders can write factors, backtest, and construct portfolios

Phase 3: Execution & Audit (Months 5–6)

Modules 6–7 (Order Execution, Audit Log):

- Multi-gate execution framework with regime/risk/microstructure checks
- Paper trading mode (simulated orders) with execution analytics
- Immutable audit trail with LLM-generated explanations (Agents 2, 4)
- Compliance dashboard and rule checklist
- Deliverable: Full-chain trading workflow with audit & compliance

Phase 4: Production Hardening (Months 7–8, ongoing)

- Real money trading (with risk limits)
- Performance optimization (latency targets: p95 <100ms per module)
- Scalability testing (1000 tickers, 10 concurrent users, 100k events/day)
- Security audit and compliance certification (if required)

Technology Reference

Open-Source Stack

- FastAPI (Python async web framework)
- React 18 + Next.js 14 (frontend)
- Nivo / D3.js (charting)
- Monaco Editor (code editing)
- TanStack Query (server state)
- Socket.IO (real-time WebSocket)
- SQLite (local state)
- yfinance (market data)

ML/Research Libraries

- scikit-learn (feature engineering, model training)
- hmmlearn (Hidden Markov Models)
- pandas / NumPy (data manipulation)
- backtrader / Zipline (backtesting engines)
- XGBoost (gradient boosting)

Academic & Reference

- Fama–French factors (momentum, value, size, profitability, investment)
- Mean reversion and reversal literature (Jegadeesh & Titman)
- Kelly Criterion (risk-optimal sizing)
- Expected Calibration Error (confidence calibration)
- Sharpe Ratio, Information Coefficient, Max Drawdown (risk metrics)

Appendix: Glossary

Term	Definition
Alpha	Excess return above a benchmark (e.g., strategy return - SPY return)
Backtest	Simulation of a strategy over historical data to evaluate performance
Calibration	How well a model's confidence matches realized accuracy
Candle	One bar on a candlestick chart (OHLCV: open, high, low, close, volume)
CVaR	Conditional Value at Risk; average loss in worst-case scenarios
DSL	Domain-Specific Language; simplified language for expressing factors
ECE	Expected Calibration Error; measure of confidence calibration
Factor	Computed feature that predicts stock returns (e.g., momentum, volatility)
Fill	Execution of an order at a market price
Gate	Execution check (regime, risk, microstructure) that must pass before order is sent
HMM	Hidden Markov Model;

	probabilistic model of market regimes
IC	Information Coefficient; correlation between factor and future returns
ICIR	IC / Std(IC); information ratio of factor
Kelly Criterion	Formula for optimal position sizing given edge and odds
Latency	Time delay from input to output (e.g., signal to execution)
LLM	Large Language Model; AI for text generation (GPT-4, etc.)
Max DD	Maximum Drawdown; largest peak-to-trough decline
Regime	Market state (trending-up, trending-down, mean-revert, crisis)
Sharpe	Sharpe Ratio; risk-adjusted return metric (annual return / annual vol)
Slippage	Difference between intended and realized fill price
Turnover	Portfolio churn rate (rebalancing frequency and magnitude)
VaR	Value at Risk (95%); potential loss in worst 5% of days
Volatility	Standard deviation of returns; measure of risk
z-score	Standardized value: (x - mean) / std; units of standard deviations